



**WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA**
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: INFORMATYKA

Jakub Szerszeń
Nr albumu studenta 71365

"Programowanie Obiektowe – System zarządzania rezerwacjami hotelowymi"

Prowadzący: mgr inż. Ewa Żesławska

Projekt

Rzeszów 2025/2026

Spis treści

Wstęp	4
1 Opis struktury projektu	5
1.1 Opis klas	5
1.2 Opis techniczny projektu	6
1.2.1 Język i narzędzia	6
1.2.2 Zarządzanie danymi	6
1.2.3 Minimalne wymagania sprzętowe	6
2 Harmonogram Realizacji projektu	7
2.1 Diagram Ganta	7
3 Repozytorium i system kontroli wersji	8
3.1 Lokalizacja repozytorium	8
4 Warstwa użytkowa	9
4.1 Interfejs menu głównego	9
4.2 Prezentacja CRUD na Tabeli Goście	9
4.3 Dodawanie Rezerwacji	10
5 Podsumowanie	13
5.1 Podsumowanie zrealizowanych prac	13
5.2 Planowane dalsze prace rozwojowe	13
Bibliografia	14
Spis rysunków	15
Streszczenie	16

Wstęp

Głównym celem projektu jest zaprojektowanie i zaimplementowanie systemu, który będzie służył do kompleksowego zarządzania rezerwacjami hotelowymi. Program będzie pozwalał na dodawanie, usuwanie, przeglądanie i modyfikowanie informacji o gościach, pokojach i rezerwacjach. Aplikacja umożliwi przypisywanie gości do pokoi w konkretnych terminach, zapewniając przy tym mechanizmy kontroli, aby rezerwacje nie nakładały się na siebie.

Wymagania funkcjonalne:

- **Obsługa bazy danych (CRUD):** Możliwość dodawania, wyświetlania, edytowania oraz usuwania rekordów dotyczących gości, pokoi i rezerwacji w relacyjnej bazie danych.
- **Zarządzanie rezerwacjami:** Funkcja przypisywania gościa do pokoju w określonym przedziale czasowym z automatycznym sprawdzeniem dostępności terminów.
- **Kategoryzacja pokoi:** Obsługa różnych standardów zakwaterowania (np. pokój standardowy, pokój Vip) przy użyciu hierarchii klas i odpowiednich tabel w bazie.

Wymagania niefunkcjonalne:

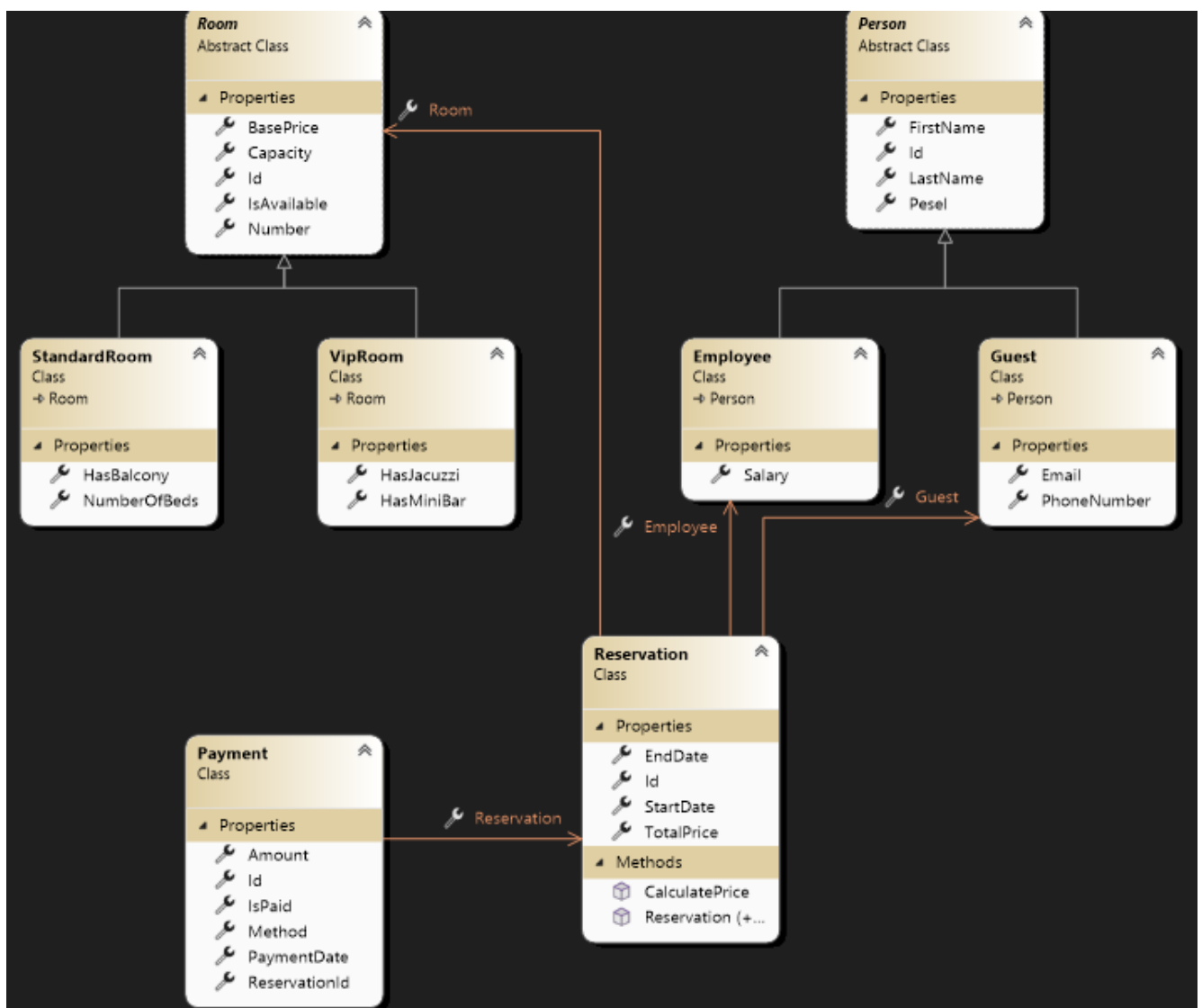
- **Trwałość danych:** Przechowywanie wszystkich informacji w relacyjnej bazie danych MSSQL, co gwarantuje spójność i bezpieczeństwo danych.
- **Bezpieczeństwo i poprawność:** Walidacja danych wejściowych (np. sprawdzanie formatu daty, numeru telefonu) przed wysłaniem zapytania do bazy danych.
- **Stabilność aplikacji:** Obsługa wyjątków, w tym zabezpieczenie przed błędami połączenia z serwerem bazy danych.
- **Architektura obiektowa:** Implementacja systemu w oparciu o 8 klas, z czego 6 klas tworzy logiczną hierarchię dziedziczenia.

Rozdział 1

Opis struktury projektu

Projekt został oparty na paradygmacie programowania obiektowego z wykorzystaniem mechanizmu dziedziczenia. Struktura składa się z 8 kluczowych klas, które pozwalają na odzwierciedlenie rzeczywistego funkcjonowania hotelu.

1.1 Opis klas



Rysunek 1.1: Diagram Klas

- **Person** (klasa abstrakcyjna): Zawiera podstawowe dane osobowe (imię, nazwisko, PESEL).
- **Guest** (dziedziczy po Person): Reprezentuje klienta hotelu, przechowuje numer telefonu i adres e-mail.
- **Employee** (dziedziczy po Person): Reprezentuje personel, zawiera informacje o stanowisku.
- **Room** (klasa abstrakcyjna): Definiuje dane pokoju takie jak: pojemność, numer, cena.
- **StandardRoom** (dziedziczy po Room): Reprezentuje pokoje o podstawowym standardzie.
- **VipRoom** (dziedziczy po Room): Reprezentuje pokoje o podwyższonym standardzie z dodatkowymi polami.
- **Reservations**: Klasa łącząca konkretnego gościa z pokojem w danym przedziale czasowym.
- **Payment**: Klasa odpowiedzialna za zarządzanie płatnościami. Łączy się z reservations po ID rezerwacji

1.2 Opis techniczny projektu

1.2.1 Język i narzędzia

System został stworzony w języku **C#** jako aplikacja konsolowa w środowisku **.NET**. Do realizacji projektu wykorzystano środowisko **Microsoft Visual Studio 2022**. Kod źródłowy jest zarządzany przez system kontroli wersji **Git**.

1.2.2 Zarządzanie danymi

Dane dotyczące rezerwacji, gości oraz pokoi są przechowywane w relacyjnej bazie danych **Microsoft SQL Server (MSSQL)**. Aplikacja realizuje wszystkie operacje **CRUD** (Create, Read, Update, Delete), co zapewnia:

- **Trwałość**: Dane nie zostają utracone po zamknięciu aplikacji.
- **Integralność**: Relacje SQL nie pozwalają na np. usunięcie pokoju, który jest obecnie zarezerwowany.
- **Stabilność**: Wykorzystanie obsługi wyjątków przy operacjach chroni program przed nagłym zamknięciem w przypadku błędów połączenia.

1.2.3 Minimalne wymagania sprzętowe

Aplikacja jest zoptymalizowana pod kątem niskiego zużycia zasobów, wymagane są następujące elementy:

- **Procesor**: min. 1.3 GHz (x64).
- **RAM**: 2 GB (wolne).
- **System**: Windows 11.
- **Baza Danych**: Microsoft SQL Server (LocalDB / Express).
- **Dysk**: 60MB

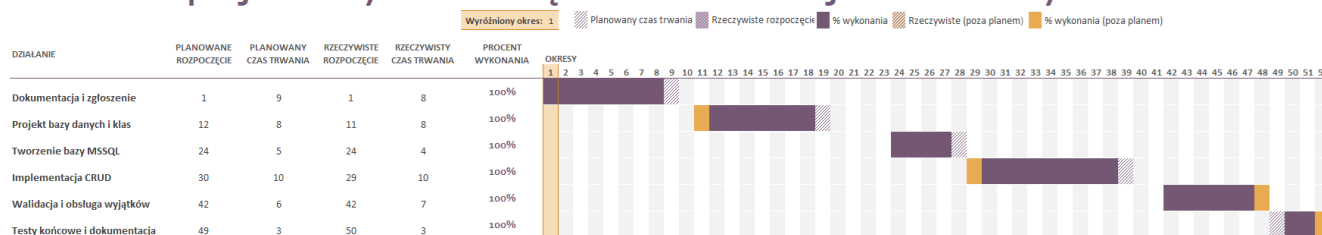
Rozdział 2

Harmonogram Realizacji projektu

W sekcji poniżej przedstawiono proces powstawania systemu zarządzania hotelowego, od momentu deklaracji tematu poprzez tworzenie całego kodu, aż do końcowej dokumentacji.

2.1 Diagram Ganta

Terminarz projektu: System Zarządzania Rezerwacjami Hotelowymi



Rysunek 2.1: Diagram Ganta – Harmonogram realizacji systemu zarządzania rezerwacjami

Proces powstawania systemu został podzielony na kilka kluczowych faz, co pozwoliło na tworzenie projektu bez presji czasu:

- **Dokumentacja i Zgłoszenie (Okresy 1-8):** Etap ten obejmował wybranie tematu, zgłoszenie oraz stworzenie początkowych stron dokumentacji, w których należało opisać założenia i cele projektu.
- **Projekt bazy danych i klas (Okresy 11-18):** Opracowano strukturę 8 klas. Powstał diagram Klas oraz struktura bazy danych.
- **Tworzenie bazy MSSQL (Okresy 24-28):** Stworzono relacyjną bazę danych Microsoft SQL Server (MSSQL) wraz z wszystkimi potrzebnymi tabelami.
- **Implementacja CRUD (Okresy 29-38):** Etap w którym zaprogramowano wszystkie klasy wraz z działaniami umożliwiającymi dodawanie, usuwanie, modyfikacje i przeglądanie wszystkich tabel w bazie danych z okna aplikacji.
- **Walidacja i obsługa wyjątków (Okresy 42-48):** Faza, w której dodano walidację i obsługę wyjątków. Działania miały na celu zapewnienie stabilności działania systemu.
- **Testy końcowe i dokumentacja (Okresy 50-52):** Przeprowadzono finalne testy poprawności działania aplikacji oraz zakończenie prac nad dokumentacją w systemie Latex.

Rozdział 3

Repozytorium i system kontroli wersji

Do zarządzania kodem został wykorzystany system Git, projekt został upubliczniony na platformie GitHub. Dzięki temu struktura plików jest przejrzysta i można łatwo przeanalizować, jak aplikacja została zaprogramowana.

3.1 Lokalizacja repozytorium

Kod źródłowy aplikacji (pliki C#) oraz skrypty potrzebne do odtworzenia bazy danych SQL są dostępne pod adresem: <https://github.com/kubaszerszen/ProgramowanieObiektowe.git>

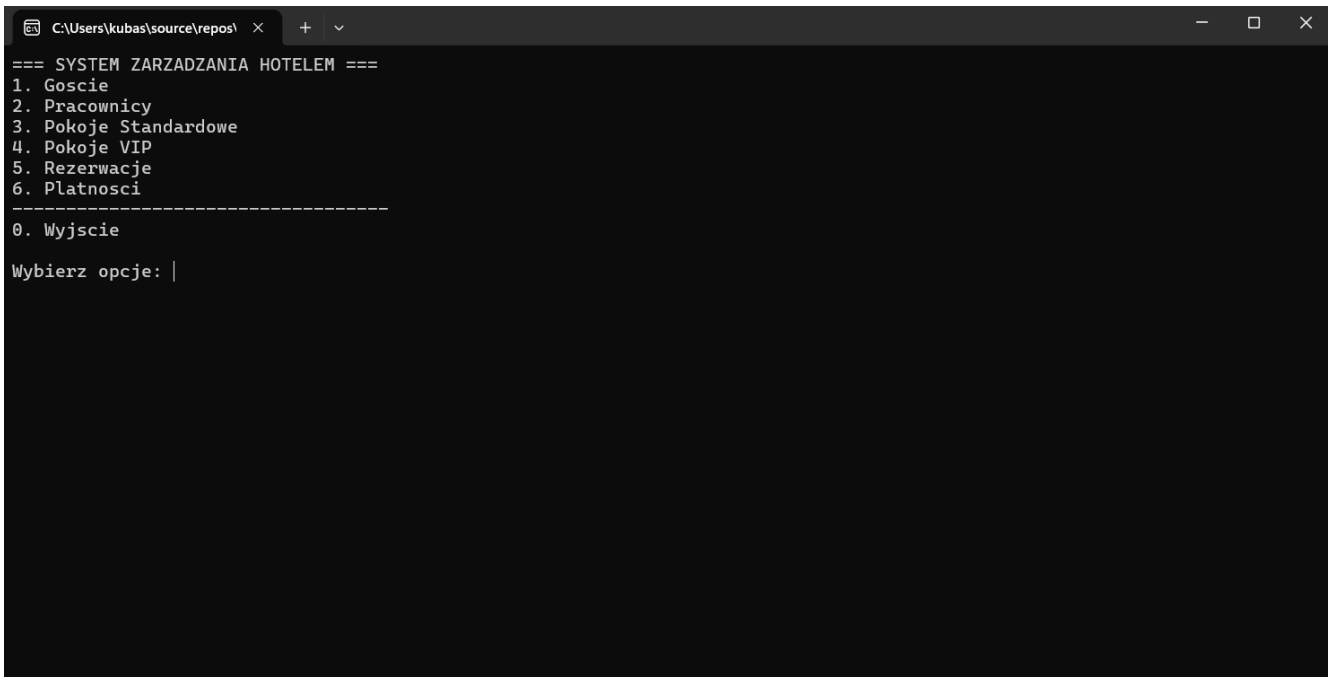
Rozdział 4

Warstwa użytkowa

Warstwa użytkowa została zaprogramowana w formie interfejsu konsolowego, umożliwia ona swobodne poruszanie się po każdej z tabel w bazie danych, pozwala również na pełne zarządzanie tymi danymi.

4.1 Interfejs menu głównego

Po uruchomieniu aplikacji, użytkownik musi dokonać wyboru w której tabeli chce dokonać zmian. w przypadku wyboru opcji **0**, program zakończy swoje działanie.



```
=== SYSTEM ZARZADZANIA HOTELEM ===
1. Goście
2. Pracownicy
3. Pokoje Standardowe
4. Pokoje VIP
5. Rezerwacje
6. Płatności
-----
0. Wyjście
Wybierz opcje: |
```

Rysunek 4.1: Interfejs menu głównego

4.2 Prezentacja CRUD na Tabeli Goście

System pozwala na wykonywanie wszystkich działań CRUD, jednocześnie weryfikując poprawność wprowadzanych danych.

```
--- ZARZADZANIE GOSCI ---
1. Lista (Wszystkie dane)
2. Dodaj goscia
3. Edytuj goscia
4. Usun goscia
0. Powrot

Wybierz: 2
Imie: Jakub
Nazwisko: Szerszen
Pesel: 06212304513
Telefon: 644332665
Email: kuba@gmail.com
SUKCES: Dodano goscia.
```

```
--- ZARZADZANIE GOSCI ---
1. Lista (Wszystkie dane)
2. Dodaj goscia
3. Edytuj goscia
4. Usun goscia
0. Powrot

Wybierz: 3
Podaj ID do edycji: 8
Nowe Imie: Jakub
Nowe Nazwisko: Buk
Pesel: 06212304514
Telefon: 794423563
Nowy Email: Jakub@gmail.com
SUKCES: Zaktualizowano dane.
```

```
--- ZARZADZANIE GOSCI ---
1. Lista (Wszystkie dane)
2. Dodaj goscia
3. Edytuj goscia
4. Usun goscia
0. Powrot

Wybierz: 4
Podaj ID do usuniecia: 8
SUKCES: Usunieto goscia.
```

```
LISTA GOSCI:
ID: 4 | julka somon | PESEL: 10222455567 | Tel: 565000988 | Email: julka@gmail.com
ID: 5 | jan pogda | PESEL: 98777765634 | Tel: 345543678 | Email: jan@gmail.com
ID: 6 | andrzej tomas | PESEL: 56708997765 | Tel: 444333222 | Email: andrzej@interia.pl
ID: 7 | Amelia Buk | PESEL: 23443223421 | Tel: 793221432 | Email: Amelia@gmail.com
ID: 8 | Jakub Szerszen | PESEL: 06212304513 | Tel: 644332665 | Email: kuba@gmail.com
```

```
LISTA GOSCI:
ID: 4 | julka somon | PESEL: 10222455567 | Tel: 565000988 | Email: julka@gmail.com
ID: 5 | jan pogda | PESEL: 98777765634 | Tel: 345543678 | Email: jan@gmail.com
ID: 6 | andrzej tomas | PESEL: 56708997765 | Tel: 444333222 | Email: andrzej@interia.pl
ID: 7 | Amelia Buk | PESEL: 23443223421 | Tel: 793221432 | Email: Amelia@gmail.com
ID: 8 | Jakub Buk | PESEL: 06212304514 | Tel: 794423563 | Email: Jakub@gmail.com
```

```
LISTA GOSCI:
ID: 4 | julka somon | PESEL: 10222455567 | Tel: 565000988 | Email: julka@gmail.com
ID: 5 | jan pogda | PESEL: 98777765634 | Tel: 345543678 | Email: jan@gmail.com
ID: 6 | andrzej tomas | PESEL: 56708997765 | Tel: 444333222 | Email: andrzej@interia.pl
ID: 7 | Amelia Buk | PESEL: 23443223421 | Tel: 793221432 | Email: Amelia@gmail.com
```

Rysunek 4.2: Działanie CRUD na tabeli Goscine

Na powyższym rysunku zaprezentowano wykonywanie wszystkich operacji CRUD:

- **Kolorem czerwonym (numer 1)** przedstawiono proces dodawania nowego gościa, a następnie pokazano listę wszystkich gości (**numer 2 czerwony**), która od razu uwzględnia nowo dodany wpis.
- **Kolorem zielonym (numer 1)** pokazano działanie procesu edycji danych istniejącego już gościa. **Zielony numer 2** prezentuje listę gości po dokonaniu zmian, gdzie widoczne są zaktualizowane dane.
- **Kolorem żółtym (numer 1)** przedstawiono procedurę usuwania gościa. Do wykonania tej operacji wymagane jest podanie jego ID, które można łatwo odczytać z listy. Na końcu pokazano listę gości po usunięciu wpisu o ID 8.

4.3 Dodawanie Rezerwacji

Przedstawiony zostanie teraz proces dodania rezerwacji oraz pól dla całkowicie nowych danych.

The image displays three separate terminal windows, each showing a menu-driven application interface. The first window, titled 'ZARZADZANIE GOSCIAMI', shows a menu with options 1-5 and 0. Option 2 is selected, and the user enters details for a guest named Jakub, resulting in a success message. A red box with the number '1' is overlaid on the menu. The second window, titled 'ZARZADZANIE PRACOWNIKAMI', shows a similar menu. Option 2 is selected, and the user enters details for an employee named Józef, resulting in a success message. A green box with the number '2' is overlaid on the menu. The third window, titled 'POKOJE VIP', shows a menu with options 1-4 and 0. Option 2 is selected, and the user enters details for a VIP room (number 304, capacity 6, base price 800), resulting in a success message. A blue box with the number '3' is overlaid on the menu.

```
--- ZARZADZANIE GOSCIAMI ---
1. Lista (Wszystkie dane)
2. Dodaj gościa
3. Edytuj gościa
4. Usun gościa
0. Powrot

Wybierz: 2
Imie: Jakub
Nazwisko: Szerszen
Pesel: 06212304514
Telefon: 445332564
Email: Jakub@gmail.com
SUKCES: Dodano gościa.

--- ZARZADZANIE PRACOWNIKAMI ---
1. Lista (Wszystkie dane)
2. Dodaj pracownika
3. Edytuj pracownika
4. Usun pracownika
0. Powrot

Wybierz: 2
Imie: Józef
Nazwisko: Bors
Pesel: 01242538920
Pensja: 6000
SUKCES: Dodano pracownika.

--- POKOJE VIP ---
1. Lista (Wszystkie dane)
2. Dodaj pokój VIP
3. Edytuj pokój VIP
4. Usun pokój VIP
0. Powrot

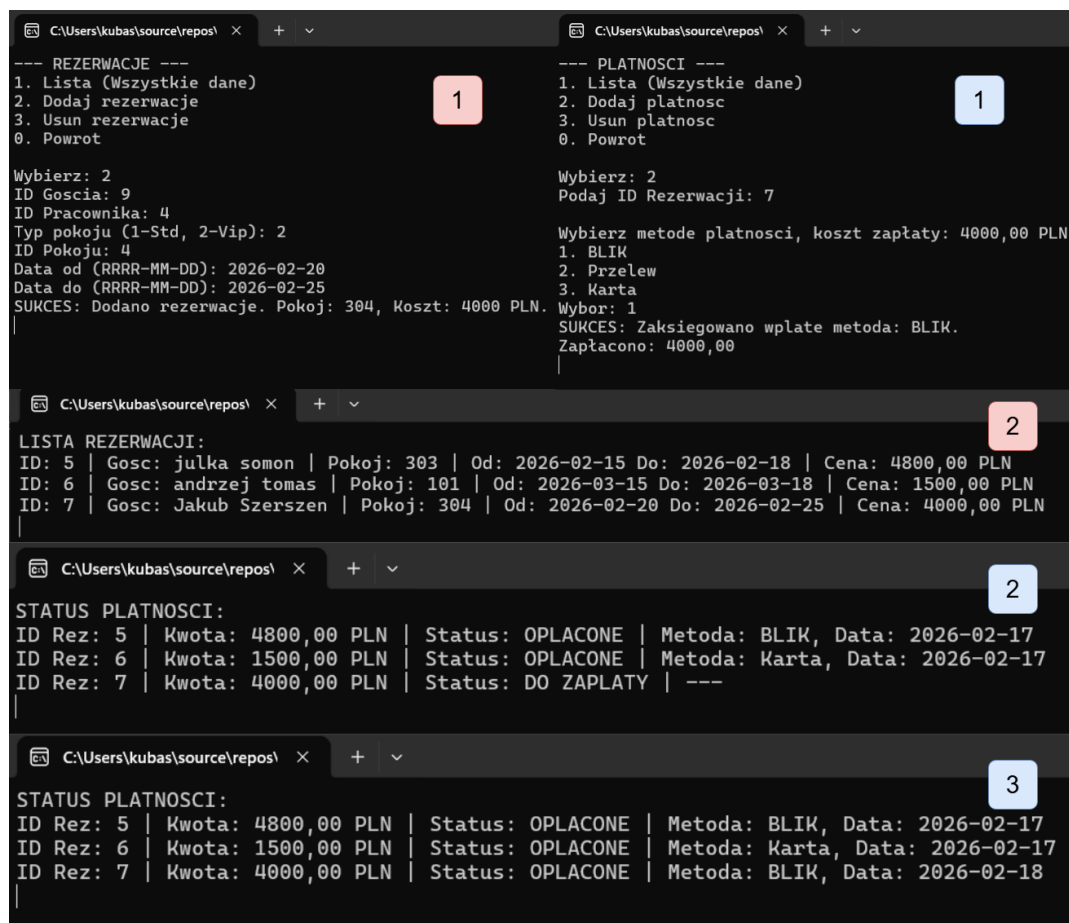
Wybierz: 2
Numer pokoju: 304
Pojemnosc: 6
Cena bazowa: 800
Jacuzzi (1-Tak, 0-Nie): 1
MiniBar (1-Tak, 0-Nie): 0
SUKCES: Dodano pokój VIP.
```

Rysunek 4.3: Dodanie nowych wierszy

Na zaprezentowanym rysunku przedstawiono proces tworzenia nowych danych, które zostaną wykorzystane do przeprowadzenia pełnej rezerwacji:

- **Tabela Goście (Rys. czerwony 1):** Została zasilona dodatkowym wierszem zawierającym unikalne dane klienta. System przypisał rekordowi kolejne wolne **ID: 9**.
- **Tabela Pracownicy (Rys. zielony 2):** Dodano nowego pracownika do systemu. Pracownikowi przypisano **ID: 4**.
- **Tabela Pokoje VIP (Rys. niebieski 3):** Skonfigurowano nowy pokój o standardzie VIP. Zdefiniowano w nim cenę bazową za noc w wysokości **800** oraz maksymalną pojemność osób wynoszącą **6 osób**. System przypisał tej jednostce **ID: 4**.

Po przygotowaniu bazy danych, przystąpiono do dokonywania rezerwacji, a finalnie opłacenia płatności.



Rysunek 4.4: Dokonanie rezerwacji i płatności

1. Utworzenie nowej rezerwacji (Rys. czerwony 1)

Proces rezerwacji polegał na powiązaniu wcześniej stworzonych obiektów:

- **ID Gościa: 9** (Jakub Szerszeń) oraz **ID Pracownika: 4** (Józef Bors).
- Wybór **pokoju VIP (ID: 4)**, co odpowiada numerowi 304 w systemie.
- Przedział czasowy od **2026-02-20** do **2026-02-25** (5 dób hotelowych).

System automatycznie wyliczył całkowity koszt pobytu na kwotę **4000 PLN** (5 dób × 800 PLN) i nadał rezerwacji unikalne **ID: 7**.

2. Rejestracja płatności (Rys. niebieski 1)

Dla nowo utworzonej rezerwacji o **ID: 7** zainicjowano moduł płatności. System pobrał należną kwotę (4000 PLN) i zaoferował wybór metody płatności. Wybrano opcję **BLIK**.

3. Weryfikacja list i płatności (Rys. czerwony 2 oraz niebieskie 2, 3)

Ostatnim etapem była kontrola integralności danych w widokach zbiorczych:

- **Lista Rezerwacji (czerwone 2):** Potwierdza obecność rezerwacji ID: 7 w pokoju 304 z poprawnie przypisaną ceną.
- **Status Płatności (niebieskie 2):** Początkowo rezerwacja ID: 7 widnieje ze statusem **DO ZAPLATY**.
- **Status Płatności (niebieskie 3):** Po finalizacji transakcji, status rezerwacji ID: 7 zmienia się na **OPLACONE**, z przypisaną metodą BLIK oraz datą operacji 2026-02-18.

Podsumowanie: Wszystkie etapy pracy zostały wykonane pomyślnie. System bez problemów zaktualizował wszystkie tabele, a dane między nimi pozostają spójne.

Rozdział 5

Podsumowanie

5.1 Podsumowanie zrealizowanych prac

W ramach projektu pomyślnie zrealizowano konsolowy system zarządzania rezerwacjami hotelowymi w języku C#. Wszystkie etapy zaplanowane w harmonogramie zostały ukończone, w tym:

- Zaprojektowano i zaimplementowano całą strukturę obiektową składającą się z 8 klas, z wykorzystaniem mechanizmu dziedziczenia oraz klas abstrakcyjnych.
- Stworzono bazę danych w systemie MSSQL, zapewniając trwałość i integralność danych.
- Wprowadzono walidację danych oraz obsługę wyjątków, co podniosło stabilność aplikacji.
- Zaimplementowano CRUD, użytkownik może dowolnie zmieniać, przeglądać, dodawać lub usuwać elementy tabel.

5.2 Planowane dalsze prace rozwojowe

Projekt posiada duży potencjał do dalszej rozbudowy:

- **Interfejs graficzny (GUI):** Przejście z aplikacji konsolowej na interfejs okienkowy, co znacznie ułatwiłoby użytkowanie.
- **Rozbudowa ochrony:** Dodanie systemu uprawnień i logowania, aby dostęp do edycji bazy danych był ograniczony tylko dla autoryzowanych użytkowników.
- **System powiadomień:** Implementacja automatycznego wysyłania potwierdzeń rezerwacji na adres e-mail gościa.
- **System płatności** Dodanie modułu, który samodzielnie będzie tworzył faktury po dokonaniu rezerwacji.
- **Płatność online:** Możliwość połączenia systemu z płatnościami online w celu umożliwienia dokonywania wpłat w czasie rzeczywistym.

Bibliografia

- [1] Microsoft, *Dokumentacja języka C#*, adres: <https://learn.microsoft.com/pl-pl/dotnet/csharp/>
- [2] Microsoft, *.NET Documentation*, adres: <https://learn.microsoft.com/pl-pl/dotnet/>
- [3] Microsoft, *Microsoft SQL Server Documentation*, adres: <https://learn.microsoft.com/en-us/sql/sql-server/>
- [4] *Materiały dydaktyczne z przedmiotu Programowanie Obiektowe*, Wykłady i prezentacje.

Spis rysunków

1.1	Diagram Klas	5
2.1	Diagram Ganta – Harmonogram realizacji systemu zarządzania rezerwacjami	7
4.1	Interfejs menu głównego	9
4.2	Działanie CRUD na tabeli Goscie	10
4.3	Dodanie nowych wierszy	11
4.4	Dokonanie rezerwacji i płatności	12

Streszczenie Projektu

"Programowanie Obiektowe – System zarządzania rezerwacjami hotelowymi"

Autor: Jakub Szerszeń

Promotor: mgr inż. Ewa Żesławska

Słowa kluczowe: C#, CRUD, MSSQL

Projekt dotyczy konsolowego systemu zarządzania rezerwacjami hotelowymi w języku C#. Aplikacja obsługuje podstawowe operacje na danych CRUD, umożliwiając dodawanie, usuwanie, przeglądanie oraz modyfikowanie informacji o gościach, pokojach i rezerwacjach. W projekcie wykorzystano dziedziczenie i hierarchię klas, co pozwala na przejrzystą strukturę oraz łatwe rozszerzanie funkcjonalności. Aplikacja sprawdza poprawność wprowadzonych danych, co pozwala unikać błędów użytkownika. System ma na celu ułatwienie zarządzania hotelem oraz sprawną obsługę gości.

The University of Information Technology and Management in Rzeszow
Faculty of Applied Information Technology

Thesis Summary

"Object-Oriented Programming – Hotel Reservation Management System"

Author: Jakub Szerszeń

Supervisor: mgr inż. Ewa Żesławska

Key words: C#, CRUD, MSSQL

The project concerns a console-based hotel reservation management system developed in C#. The application supports basic CRUD operations, allowing for adding, deleting, viewing, and modifying information about guests, rooms, and reservations. The project utilizes inheritance and a class hierarchy, providing a clear structure and easy expandability of functionality. The application also validates user input to prevent errors. The system aims to make hotel management easier and ensure efficient guest service.